

RepoAgent

Streamlining Your Documentation Workflow Like Never Before

-- Qinyu (“Logic”) Luo Nov. 2024

Email: qinyuluo123@gmail.com
qluo8@jh.edu



The story starts from **XAgent**

https://github.com/OpenBMB/XAgent/

XAgent

Public

Edit Pins | Watch 75 | Fork 825 | Starred 8k

main | 2 Branches | 1 Tags | Go to file t | Add file | Code

shouldnotappearcalm fix: word spelling problem in XAgent/data_structure/plan.py ... 3619c25 · 7 months ago 226 Commits

.github	docs: Update PULL_REQUEST_TEMPLATE.md to unrefere...	10 months ago
Markdown_Docs	docs: add markdown docs for xagent1.0, generated by ou...	9 months ago
ToolServer	fix: add cmd to install dep httpx (#353)	9 months ago
XAgent	fix: word spelling problem in XAgent/data_structure/pla...	7 months ago
XAgentGen	docs: put xagentgen docker into tool-server-network (#2...	10 months ago
XAgentServer	fix: grammar typo in XAgentServer/server.py (#348)	9 months ago
XAgentWeb	update doc	9 months ago
assets	MOD: remove ToolServerMonitor	10 months ago
dockerfiles	fix: add cmd to install dep httpx (#353)	9 months ago
local_workspace	init	last year
tests	test: add test cases for run.py using pytest. (#335)	9 months ago
.env	Squashed commit of the following:	last year
.gitignore	merged some changes	10 months ago
.project_hierarchy.json	fix: word spelling problem in XAgent/data_structure/pla...	7 months ago
CHANGELOG.md	Release v1.0.0 (#237)	10 months ago
CODE_OF_CONDUCT.md	docs: update README with new workflow diagram and do...	10 months ago
CONTRIBUTING.md	Update CONTRIBUTING.md	last year

About

An Autonomous LLM Agent for Complex Task Solving

blog.x-agent.net/blog/xagent/

Readme
Apache-2.0 license
Code of conduct
Activity
Custom properties
8k stars
75 watching
825 forks
Report repository

Releases 1

v1.0.0 Latest
on Nov 22, 2023

Packages

No packages published

Contributors 34



+ 20 contributors

Documentation Engineer

It’s hard work but super important...



Creating, Managing, and Maintaining technical documentation...

Collaborate with product managers, developers, and other team members

Can we build an AI Document Engineer ?

Goals

Clarity and Simplicity

Accuracy and Consistency

Auto Maintaining and Sychronizing

Well Structured

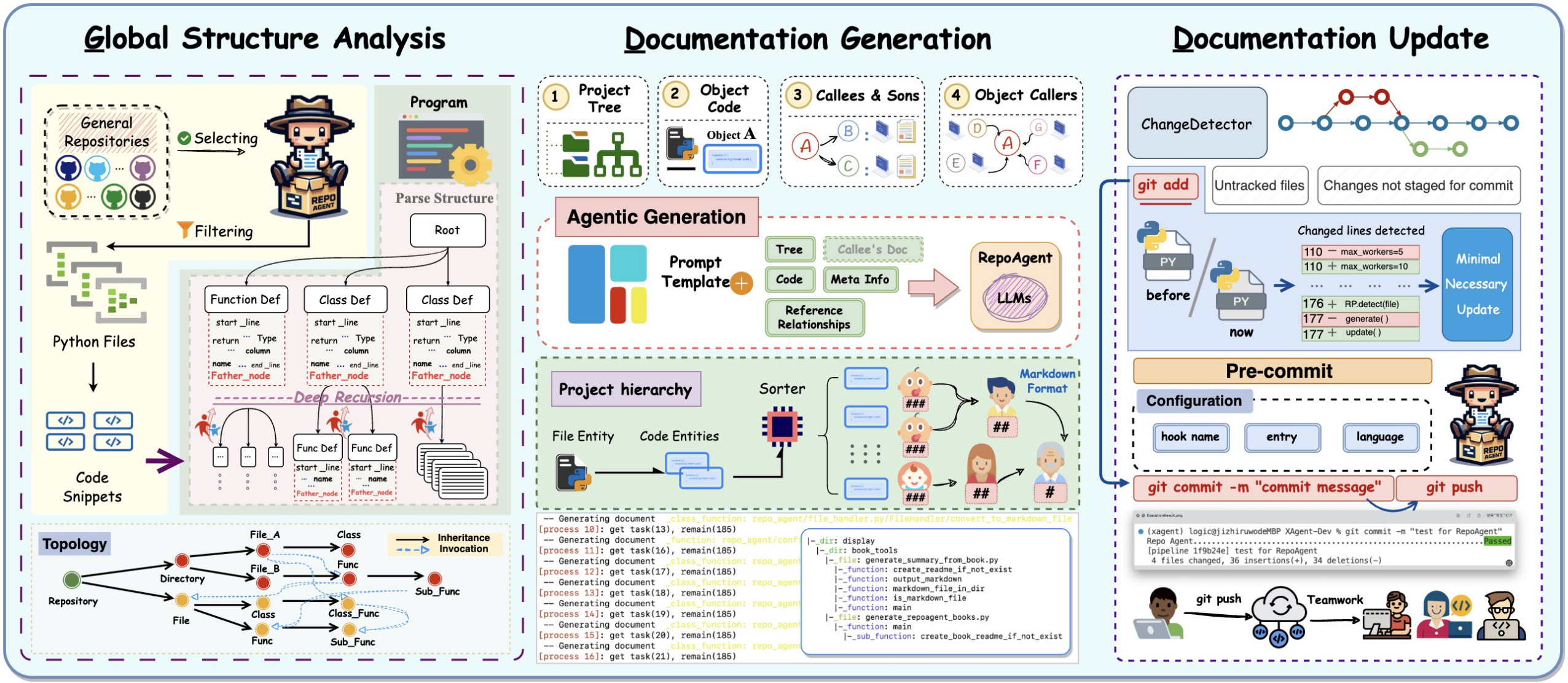
Easy to Understand (Examples)

Easy to Retrieve

Global Perception

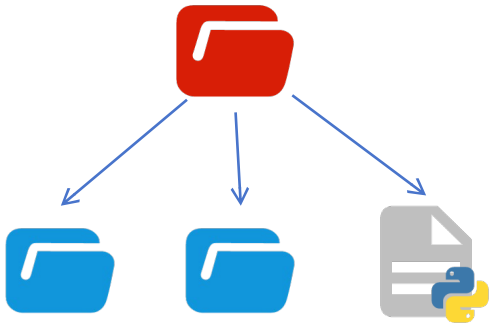
... ...

Methodology



Breakdown Structure

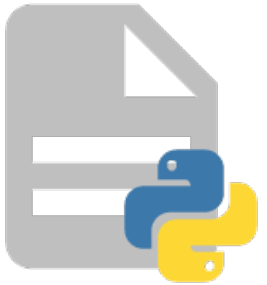
Repository-level && AST-walk



GitignoreChecker

Also can configure
ignore_list = [ ]

- ClassDef EdgeType
  - ClassDef DocItemType
    - FunctionDef to_str(self)
      - FunctionDef print_self(self)
        - FunctionDef get_edge_ty...
          - ClassDef DocItemStatus
            - ClassDef DocItem
              - FunctionDef has_ans_rela...



Code Snippets


```
{
  "setup.py": [
    {
      "type": "ClassDef",
      "name": "InstallMermaidCLI",
      "md_content": [],
      "code_start_line": 8,
      "code_end_line": 18,
      "params": [],
      "have_return": false,
      "code_content": "class InstallMermaidCLI(Command):\n
      "name_column": 6,
      "item_status": "doc_has_not_been_generated",
      "who_reference_me": [],
      "reference_who": [],
      "special_reference_type": []
    },
```

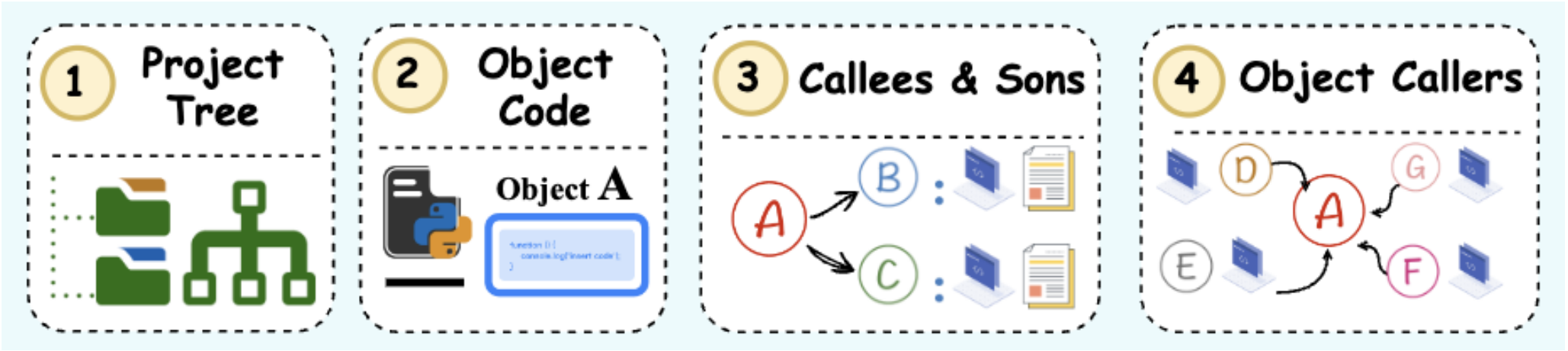
Global bi-directional callee & callers



https://jedi.readthedocs.io/en/latest/

This is great, but we can only get one direction (**script.get_references**)…

In this way, we have to parse the whole project to get “who I referenced” and record it in the global project_hierarchy



Prompt Template

LLMs

You are an AI documentation assistant, and your task is to generate documentation based on the given code of an object. The purpose of the documentation is to help developers and beginners understand the function and specific usage of the code.

Currently, you are in a project, and the hierarchical structure of this project is as follows:

`Tree`

The path to the target object in this project is:

`Meta Info - File Path`

Now you need to generate a document for a **{Code_type}**, whose name is **{Code_Name}**.

The content of the target code is as follows:

`Code`

And please include the reference relationship with its callers and callees in the project from a functional perspective

`Reference Relationships`

`Documentation`

Please generate a detailed explanation document for this object based on the code of the target object itself and combine it with its calling situation in the project.

Please write out the function of this **{Code_type}** in bold plain text, followed by a detailed analysis in plain text (including all details), in language **{language}** to serve as the documentation for this part of the code.

The STANDARD FORMAT is as follows:

{Code_Name}: The function of this **{Code_type}** is XXX
(Detailed code analysis and description...)

{Parameters_or_Attribute}: The **{parameters_or_attribute}** of this **{Code_type}**.
- parameters-1: XXX
- ... ...

Note: Points to note about the use of the code

Output Example: Mock up a possible appearance of the code's return value.

Please note:
- Any part of the content...
- Write mainly in ...
- ... ...

Project Tree

File hierarchy

Object Meta Information

Code

Bi-directional relationship

Related former code docs

What if length exceed or other error...

Auto model switching

Auto info shrinking

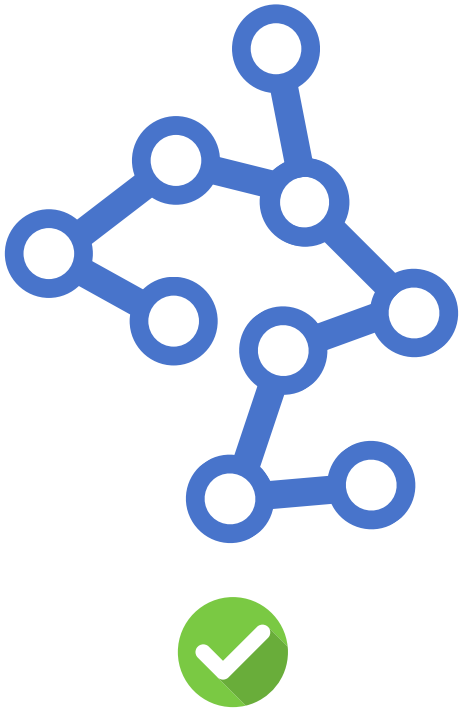
Exception handling and retry method

Summarizing

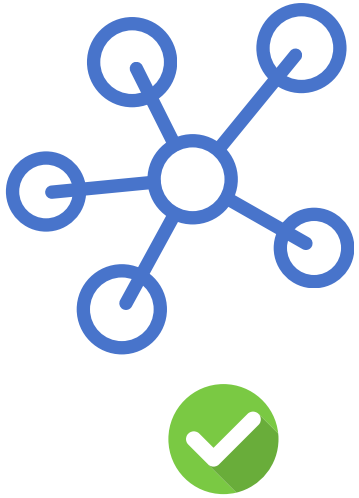
Chunking

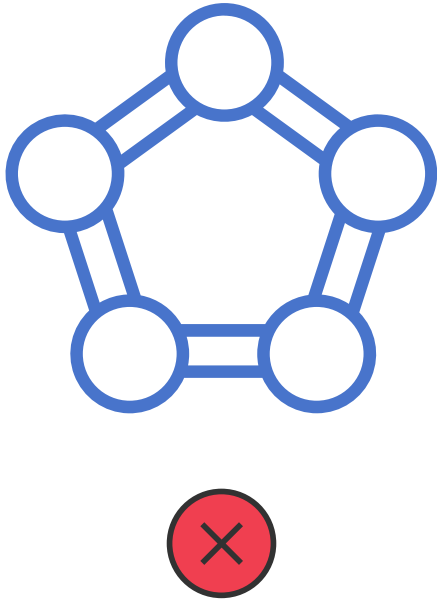
.....

Topology Generation



task_available_func will ensure that the tasks of the dependent objects have been identified and processed before adding the tasks of this object.





We make sure leaf node should be generated at first. This is helpful if users want more accurate and relavent results by putting callers/callees documentation into the perception.

输入并搜索

XAgent
RepoAgent

Introduction
chatdev
utils
roster
statistics
phase
chat_env
composed_phase
chat_chain
codes
documents
camel
model_backend
messages
generators
web_spider
utils
agents
critic_agent
task_agent
tool_agents
base
hugging_face_tool_agent

ClassDef TaskSpecifyAgent

TaskSpecifyAgent: The function of TaskSpecifyAgent is to refine a given task prompt by asking the user to provide more details, thereby creating a more specific task description.

Attributes:

- `DEFAULT_WORD_LIMIT` : The default word limit for the task prompt, set to 50.
- `task_specify_prompt` : The prompt used for specifying the task, which can be a string or an instance of TextPrompt.

Code Description: The TaskSpecifyAgent class is a subclass of ChatAgent and is designed to interact with users to specify a given task prompt. It inherits the properties and methods from ChatAgent, which manages conversation flows and generates responses using a language model.

The constructor of TaskSpecifyAgent accepts several parameters:

- `model` : The type of language model to use for the agent. It defaults to `ModelType.GPT_3_5_TURBO` .
- `task_type` : The type of task for which to generate a prompt. It defaults to `TaskType.AI_SOCIETY` .
- `model_config` : The configuration for the model. If not provided, a default configuration with a temperature of 1.0 is used.
- `task_specify_prompt` : An optional prompt for specifying the task. If not provided, a default prompt template is generated based on the task type.
- `word_limit` : The word limit for the task prompt, defaulting to `DEFAULT_WORD_LIMIT` .

The `__init__` method initializes the task_specify_prompt either by using the provided prompt or by generating one based on the task type and word limit. It then constructs a system message that describes the role of the agent as "Task Specifier" and initializes the parent ChatAgent with this system message, the model, and the model configuration.

The `step` method is the core function of the TaskSpecifyAgent. It takes an original task prompt and an optional dictionary of additional information (meta_dict) and uses these to format the task_specify_prompt. The method then resets the agent's state, sends the formatted task_specify_prompt as a user message to the ChatAgent's step method, and processes the response. If the response is empty or the conversation is terminated, a RuntimeError is raised. Otherwise, it returns a TextPrompt containing the specified task prompt.

Note:

- The TaskSpecifyAgent class is part of the CAMEL framework and is specifically designed to work within this context.
- The class is intended to be used in scenarios where task prompts need to be refined for clarity and specificity.
- The class relies on the proper functioning of the ChatAgent class and its methods, particularly the `reset` and `step` methods.
- The class assumes that the language model used is capable of generating coherent and contextually relevant responses based on the task prompts provided.

Output Example:

```
# Assuming the existence of a TaskSpecifyAgent instance named `task_specifier`.
specified_prompt = task_specifier.step("Write a poem about the sea.")
# The output is a TextPrompt object containing the specified task prompt.
```

Copy

输入并搜索

XAgent

RepoAgent

Introduction

chatdev

utils

roster

statistics

phase

chat_env

composed_phase

chat_chain

codes

documents

camel

model_backend

messages

generators

web_spider

utils

agents

critic_agent

task_agent

tool_agents

base

hugging_face_tool_agent

ClassDef TaskSpecifyAgent

TaskSpecifyAgent: The function of TaskSpecifyAgent is to refine a given task prompt by asking the user to provide more details, thereby creating a more specific task description.

Attributes:

- `DEFAULT_WORD_LIMIT` : The default word limit for the task prompt, set to 50.
- `task_specify_prompt` : The prompt used for specifying the task, which can be a string or an instance of TextPrompt.

Code Description: The TaskSpecifyAgent class is a subclass of ChatAgent and is designed to interact with users to specify a given task prompt. It inherits the properties and methods from ChatAgent, which manages conversation flows and generates responses using a language model.

The constructor of TaskSpecifyAgent accepts several parameters:

- `model` : The type of language model to use for the agent. It defaults to `ModelType.GPT_3_5_TURBO` .
- `task_type` : The type of task for which to generate a prompt. It defaults to `TaskType.AI_SOCIETY` .
- `model_config` : The configuration for the model. If not provided, a default configuration with a temperature of 1.0 is used.
- `task_specify_prompt` : An optional prompt for specifying the task. If not provided, a default prompt template is generated based on the task type.
- `word_limit` : The word limit for the task prompt, defaulting to `DEFAULT_WORD_LIMIT` .

The `__init__` method initializes the task_specify_prompt either by using the provided prompt or by generating one based on the task type and word limit. It then constructs a system message that describes the role of the agent as "Task Specifier" and initializes the parent ChatAgent with this system message, the model, and the model configuration.

The `step` method is the core function of the TaskSpecifyAgent. It takes an original task prompt and an optional dictionary of additional information (meta_dict) and uses these to format the task_specify_prompt. The method then resets the agent's state, sends the formatted task_specify_prompt as a user message to the ChatAgent's step method, and processes the response. If the response is empty or the conversation is terminated, a RuntimeError is raised. Otherwise, it returns a TextPrompt containing the specified task prompt.

Note:

- The TaskSpecifyAgent class is part of the CAMEL framework and is specifically designed to work within this context.
- The class is intended to be used in scenarios where task prompts need to be refined for clarity and specificity.
- The class relies on the proper functioning of the ChatAgent class and its methods, particularly the `reset` and `step` methods.
- The class assumes that the language model used is capable of generating coherent and contextually relevant responses based on the task prompts provided.

Output Example:

Copy

```
# Assuming the existence of a TaskSpecifyAgent instance named `task_specifier`.
specified_prompt = task_specifier.step("Write a poem about the sea.")
# The output is a TextPrompt object containing the specified task prompt.
```

输入并搜索

XAgent
RepoAgent

Introduction
chatdev
utils
roster
statistics
phase
chat_env
composed_phase
chat_chain
codes
documents
camel
model_backend
messages
generators
web_spider
utils
agents
critic_agent
task_agent
tool_agents
base
hugging_face_tool_agent

ClassDef TaskSpecifyAgent

TaskSpecifyAgent: The function of TaskSpecifyAgent is to refine a given task prompt by asking the user to provide more details, thereby creating a more specific task description.

Attributes:

- `DEFAULT_WORD_LIMIT` : The default word limit for the task prompt, set to 50.
- `task_specify_prompt` : The prompt used for specifying the task, which can be a string or an instance of TextPrompt.

Code Description: The TaskSpecifyAgent class is a subclass of ChatAgent and is designed to interact with users to specify a given task prompt. It inherits the properties and methods from ChatAgent, which manages conversation flows and generates responses using a language model.

The constructor of TaskSpecifyAgent accepts several parameters:

- `model` : The type of language model to use for the agent. It defaults to `ModelType.GPT_3_5_TURBO` .
- `task_type` : The type of task for which to generate a prompt. It defaults to `TaskType.AI_SOCIETY` .
- `model_config` : The configuration for the model. If not provided, a default configuration with a temperature of 1.0 is used.
- `task_specify_prompt` : An optional prompt for specifying the task. If not provided, a default prompt template is generated based on the task type.
- `word_limit` : The word limit for the task prompt, defaulting to `DEFAULT_WORD_LIMIT` .

The `__init__` method initializes the task_specify_prompt either by using the provided prompt or by generating one based on the task type and word limit. It then constructs a system message that describes the role of the agent as "Task Specifier" and initializes the parent ChatAgent with this system message, the model, and the model configuration.

The `step` method is the core function of the TaskSpecifyAgent. It takes an original task prompt and an optional dictionary of additional information (meta_dict) and uses these to format the task_specify_prompt. The method then resets the agent's state, sends the formatted task_specify_prompt as a user message to the ChatAgent's step method, and processes the response. If the response is empty or the conversation is terminated, a RuntimeError is raised. Otherwise, it returns a TextPrompt containing the specified task prompt.

Note:

- The TaskSpecifyAgent class is part of the CAMEL framework and is specifically designed to work within this context.
- The class is intended to be used in scenarios where task prompts need to be refined for clarity and specificity.
- The class relies on the proper functioning of the ChatAgent class and its methods, particularly the `reset` and `step` methods.
- The class assumes that the language model used is capable of generating coherent and contextually relevant responses based on the task prompts provided.

Output Example:

Copy

```
# Assuming the existence of a TaskSpecifyAgent instance named `task_specifier`.
specified_prompt = task_specifier.step("Write a poem about the sea.")
# The output is a TextPrompt object containing the specified task prompt.
```

输入并搜索

XAgent

RepoAgent

Introduction

chatdev

utils

roster

statistics

phase

chat_env

composed_phase

chat_chain

codes

documents

camel

model_backend

messages

generators

web_spider

utils

agents

critic_agent

task_agent

tool_agents

base

hugging_face_tool_agent

ClassDef TaskSpecifyAgent

TaskSpecifyAgent: The function of TaskSpecifyAgent is to refine a given task prompt by asking the user to provide more details, thereby creating a more specific task description.

Attributes:

- `DEFAULT_WORD_LIMIT` : The default word limit for the task prompt, set to 50.
- `task_specify_prompt` : The prompt used for specifying the task, which can be a string or an instance of TextPrompt.

Code Description: The TaskSpecifyAgent class is a subclass of ChatAgent and is designed to interact with users to specify a given task prompt. It inherits the properties and methods from ChatAgent, which manages conversation flows and generates responses using a language model.

The constructor of TaskSpecifyAgent accepts several parameters:

- `model` : The type of language model to use for the agent. It defaults to `ModelType.GPT_3_5_TURBO` .
- `task_type` : The type of task for which to generate a prompt. It defaults to `TaskType.AI_SOCIETY` .
- `model_config` : The configuration for the model. If not provided, a default configuration with a temperature of 1.0 is used.
- `task_specify_prompt` : An optional prompt for specifying the task. If not provided, a default prompt template is generated based on the task type.
- `word_limit` : The word limit for the task prompt, defaulting to `DEFAULT_WORD_LIMIT` .

The `__init__` method initializes the task_specify_prompt either by using the provided prompt or by generating one based on the task type and word limit. It then constructs a system message that describes the role of the agent as "Task Specifier" and initializes the parent ChatAgent with this system message, the model, and the model configuration.

The `step` method is the core function of the TaskSpecifyAgent. It takes an original task prompt and an optional dictionary of additional information (meta_dict) and uses these to format the task_specify_prompt. The method then resets the agent's state, sends the formatted task_specify_prompt as a user message to the ChatAgent's step method, and processes the response. If the response is empty or the conversation is terminated, a RuntimeError is raised. Otherwise, it returns a TextPrompt containing the specified task prompt.

Note:

- The TaskSpecifyAgent class is part of the CAMEL framework and is specifically designed to work within this context.
- The class is intended to be used in scenarios where task prompts need to be refined for clarity and specificity.
- The class relies on the proper functioning of the ChatAgent class and its methods, particularly the `reset` and `step` methods.
- The class assumes that the language model used is capable of generating coherent and contextually relevant responses based on the task prompts provided.

Output Example:

Copy

```
# Assuming the existence of a TaskSpecifyAgent instance named `task_specifier`.
specified_prompt = task_specifier.step("Write a poem about the sea.")
# The output is a TextPrompt object containing the specified task prompt.
```

输入并搜索

XAgent

RepoAgent

Introduction

chatdev

utils

roster

statistics

phase

chat_env

composed_phase

chat_chain

codes

documents

camel

model_backend

messages

generators

web_spider

utils

agents

critic_agent

task_agent

tool_agents

base

hugging_face_tool_agent

ClassDef TaskSpecifyAgent

TaskSpecifyAgent: The function of TaskSpecifyAgent is to refine a given task prompt by asking the user to provide more details, thereby creating a more specific task description.

Attributes:

- `DEFAULT_WORD_LIMIT` : The default word limit for the task prompt, set to 50.
- `task_specify_prompt` : The prompt used for specifying the task, which can be a string or an instance of TextPrompt.

Code Description: The TaskSpecifyAgent class is a subclass of ChatAgent and is designed to interact with users to specify a given task prompt. It inherits the properties and methods from ChatAgent, which manages conversation flows and generates responses using a language model.

The constructor of TaskSpecifyAgent accepts several parameters:

- `model` : The type of language model to use for the agent. It defaults to `ModelType.GPT_3_5_TURBO` .
- `task_type` : The type of task for which to generate a prompt. It defaults to `TaskType.AI_SOCIETY` .
- `model_config` : The configuration for the model. If not provided, a default configuration with a temperature of 1.0 is used.
- `task_specify_prompt` : An optional prompt for specifying the task. If not provided, a default prompt template is generated based on the task type.
- `word_limit` : The word limit for the task prompt, defaulting to `DEFAULT_WORD_LIMIT` .

The `__init__` method initializes the task_specify_prompt either by using the provided prompt or by generating one based on the task type and word limit. It then constructs a system message that describes the role of the agent as "Task Specifier" and initializes the parent ChatAgent with this system message, the model, and the model configuration.

The `step` method is the core function of the TaskSpecifyAgent. It takes an original task prompt and an optional dictionary of additional information (meta_dict) and uses these to format the task_specify_prompt. The method then resets the agent's state, sends the formatted task_specify_prompt as a user message to the ChatAgent's step method, and processes the response. If the response is empty or the conversation is terminated, a RuntimeError is raised. Otherwise, it returns a TextPrompt containing the specified task prompt.

Note:

- The TaskSpecifyAgent class is part of the CAMEL framework and is specifically designed to work within this context.
- The class is intended to be used in scenarios where task prompts need to be refined for clarity and specificity.
- The class relies on the proper functioning of the ChatAgent class and its methods, particularly the `reset` and `step` methods.
- The class assumes that the language model used is capable of generating coherent and contextually relevant responses based on the task prompts provided.

Output Example:

Copy

```
# Assuming the existence of a TaskSpecifyAgent instance named `task_specifier`.
specified_prompt = task_specifier.step("Write a poem about the sea.")
# The output is a TextPrompt object containing the specified task prompt.
```

输入并搜索

XAgent

RepoAgent

Introduction

chatdev

utils

roster

statistics

phase

chat_env

composed_phase

chat_chain

codes

documents

camel

model_backend

messages

generators

web_spider

utils

agents

critic_agent

task_agent

tool_agents

base

hugging_face_tool_agent

ClassDef TaskSpecifyAgent

TaskSpecifyAgent: The function of TaskSpecifyAgent is to refine a given task prompt by asking the user to provide more details, thereby creating a more specific task description.

Attributes:

- `DEFAULT_WORD_LIMIT` : The default word limit for the task prompt, set to 50.
- `task_specify_prompt` : The prompt used for specifying the task, which can be a string or an instance of TextPrompt.

Code Description: The TaskSpecifyAgent class is a subclass of ChatAgent and is designed to interact with users to specify a given task prompt. It inherits the properties and methods from ChatAgent, which manages conversation flows and generates responses using a language model.

The constructor of TaskSpecifyAgent accepts several parameters:

- `model` : The type of language model to use for the agent. It defaults to `ModelType.GPT_3_5_TURBO` .
- `task_type` : The type of task for which to generate a prompt. It defaults to `TaskType.AI_SOCIETY` .
- `model_config` : The configuration for the model. If not provided, a default configuration with a temperature of 1.0 is used.
- `task_specify_prompt` : An optional prompt for specifying the task. If not provided, a default prompt template is generated based on the task type.
- `word_limit` : The word limit for the task prompt, defaulting to `DEFAULT_WORD_LIMIT` .

The `__init__` method initializes the task_specify_prompt either by using the provided prompt or by generating one based on the task type and word limit. It then constructs a system message that describes the role of the agent as "Task Specifier" and initializes the parent ChatAgent with this system message, the model, and the model configuration.

The `step` method is the core function of the TaskSpecifyAgent. It takes an original task prompt and an optional dictionary of additional information (meta_dict) and uses these to format the task_specify_prompt. The method then resets the agent's state, sends the formatted task_specify_prompt as a user message to the ChatAgent's step method, and processes the response. If the response is empty or the conversation is terminated, a RuntimeError is raised. Otherwise, it returns a TextPrompt containing the specified task prompt.

Note:

- The TaskSpecifyAgent class is part of the CAMEL framework and is specifically designed to work within this context.
- The class is intended to be used in scenarios where task prompts need to be refined for clarity and specificity.
- The class relies on the proper functioning of the ChatAgent class and its methods, particularly the `reset` and `step` methods.
- The class assumes that the language model used is capable of generating coherent and contextually relevant responses based on the task prompts provided.

Output Example:

Copy

```
# Assuming the existence of a TaskSpecifyAgent instance named `task_specifier`.
specified_prompt = task_specifier.step("Write a poem about the sea.")
# The output is a TextPrompt object containing the specified task prompt.
```

输入并搜索

XAgent
RepoAgent

Introduction
chatdev
utils
roster
statistics
phase
chat_env
composed_phase
chat_chain
codes
documents
camel
model_backend
messages
generators
web_spider
utils
agents
critic_agent
task_agent
tool_agents
base
hugging_face_tool_agent

ClassDef TaskSpecifyAgent

TaskSpecifyAgent: The function of TaskSpecifyAgent is to refine a given task prompt by asking the user to provide more details, thereby creating a more specific task description.

Attributes:

- `DEFAULT_WORD_LIMIT` : The default word limit for the task prompt, set to 50.
- `task_specify_prompt` : The prompt used for specifying the task, which can be a string or an instance of TextPrompt.

Code Description: The TaskSpecifyAgent class is a subclass of ChatAgent and is designed to interact with users to specify a given task prompt. It inherits the properties and methods from ChatAgent, which manages conversation flows and generates responses using a language model.

The constructor of TaskSpecifyAgent accepts several parameters:

- `model` : The type of language model to use for the agent. It defaults to `ModelType.GPT_3_5_TURBO` .
- `task_type` : The type of task for which to generate a prompt. It defaults to `TaskType.AI_SOCIETY` .
- `model_config` : The configuration for the model. If not provided, a default configuration with a temperature of 1.0 is used.
- `task_specify_prompt` : An optional prompt for specifying the task. If not provided, a default prompt template is generated based on the task type.
- `word_limit` : The word limit for the task prompt, defaulting to `DEFAULT_WORD_LIMIT` .

The `__init__` method initializes the task_specify_prompt either by using the provided prompt or by generating one based on the task type and word limit. It then constructs a system message that describes the role of the agent as "Task Specifier" and initializes the parent ChatAgent with this system message, the model, and the model configuration.

The `step` method is the core function of the TaskSpecifyAgent. It takes an original task prompt and an optional dictionary of additional information (meta_dict) and uses these to format the task_specify_prompt. The method then resets the agent's state, sends the formatted task_specify_prompt as a user message to the ChatAgent's step method, and processes the response. If the response is empty or the conversation is terminated, a RuntimeError is raised. Otherwise, it returns a TextPrompt containing the specified task prompt.

Note:

- The TaskSpecifyAgent class is part of the CAMEL framework and is specifically designed to work within this context.
- The class is intended to be used in scenarios where task prompts need to be refined for clarity and specificity.
- The class relies on the proper functioning of the ChatAgent class and its methods, particularly the `reset` and `step` methods.
- The class assumes that the language model used is capable of generating coherent and contextually relevant responses based on the task prompts provided.

Output Example:

```
# Assuming the existence of a TaskSpecifyAgent instance named `task_specifier`.
specified_prompt = task_specifier.step("Write a poem about the sea.")
# The output is a TextPrompt object containing the specified task prompt.
```

Copy

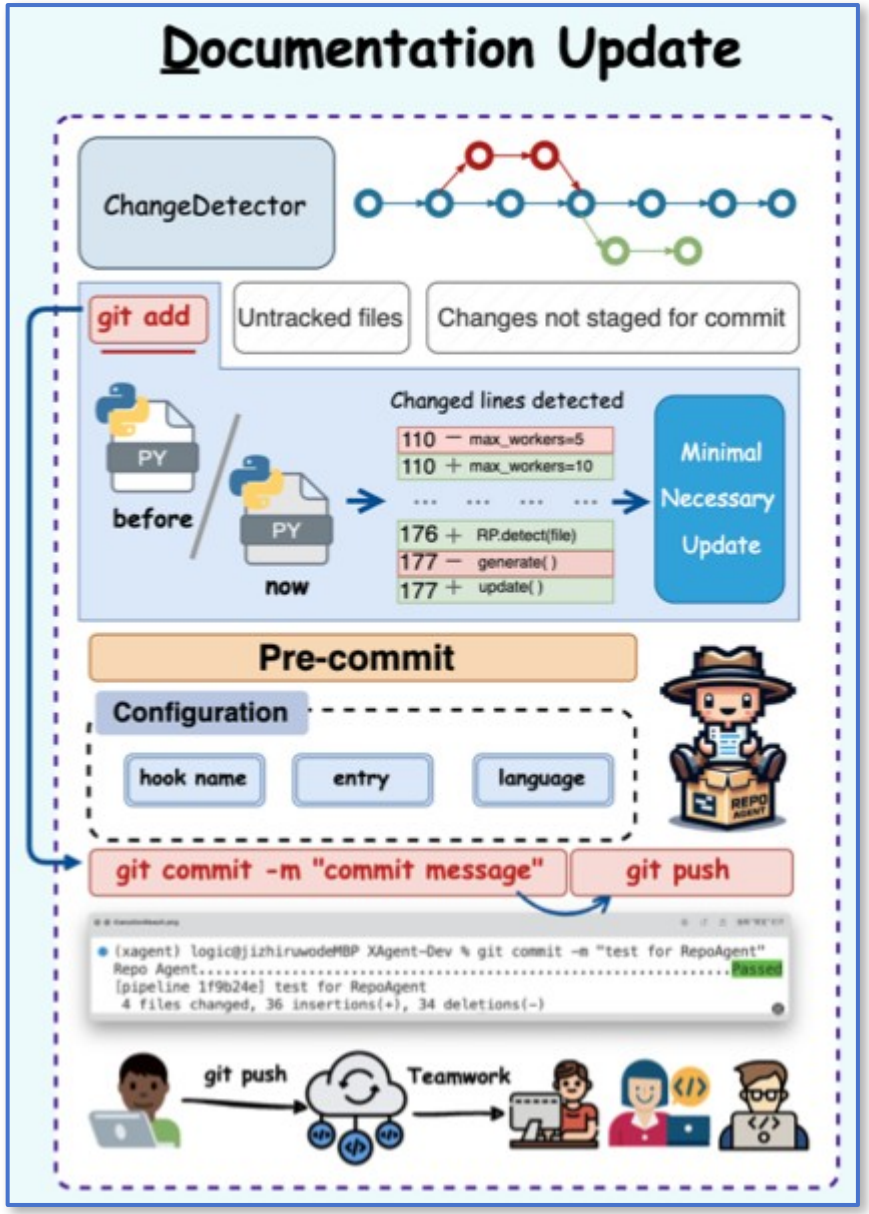
Markdown Documentation

Minimize the necessity to update
-- keep consistency

We designed RepoAgent to makes the Documentation like a book in mark down format



Detect changes and auto update



At first we use only git to detect change

We build “fake files” to ensure he documentation generation process uses the latest code changes without affecting the actual working directory state.

Avoids possible errors and human negligence.

Addresses various edge cases.

Utilizes the idea of merging

Auto Maintaining

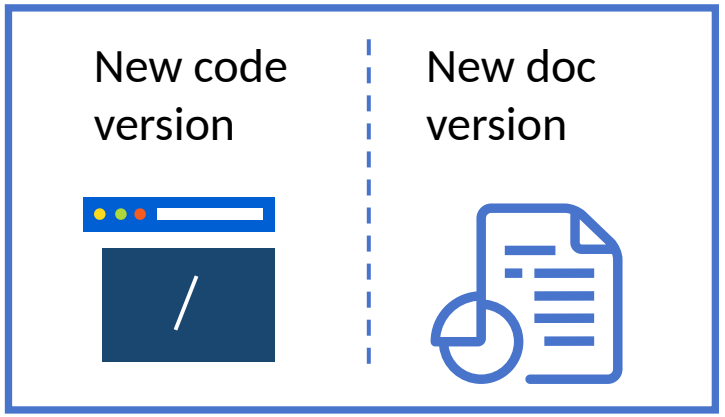
RepoAgent fully complies with the original development process and habits

Coding → **Git add** → **Git commit** → **Git push**

Auto commit by RepoAgent







simultanously update for one version

Evaluations

	Total	Human	Model	Win Rate
Transformers	150	45	105	**0.70**
LlamaIndex	150	13	137	**0.91**

Table 1: Results of human preference test on human-authored and model-generated code documentation.

Human evaluation result

Evaluation criteria

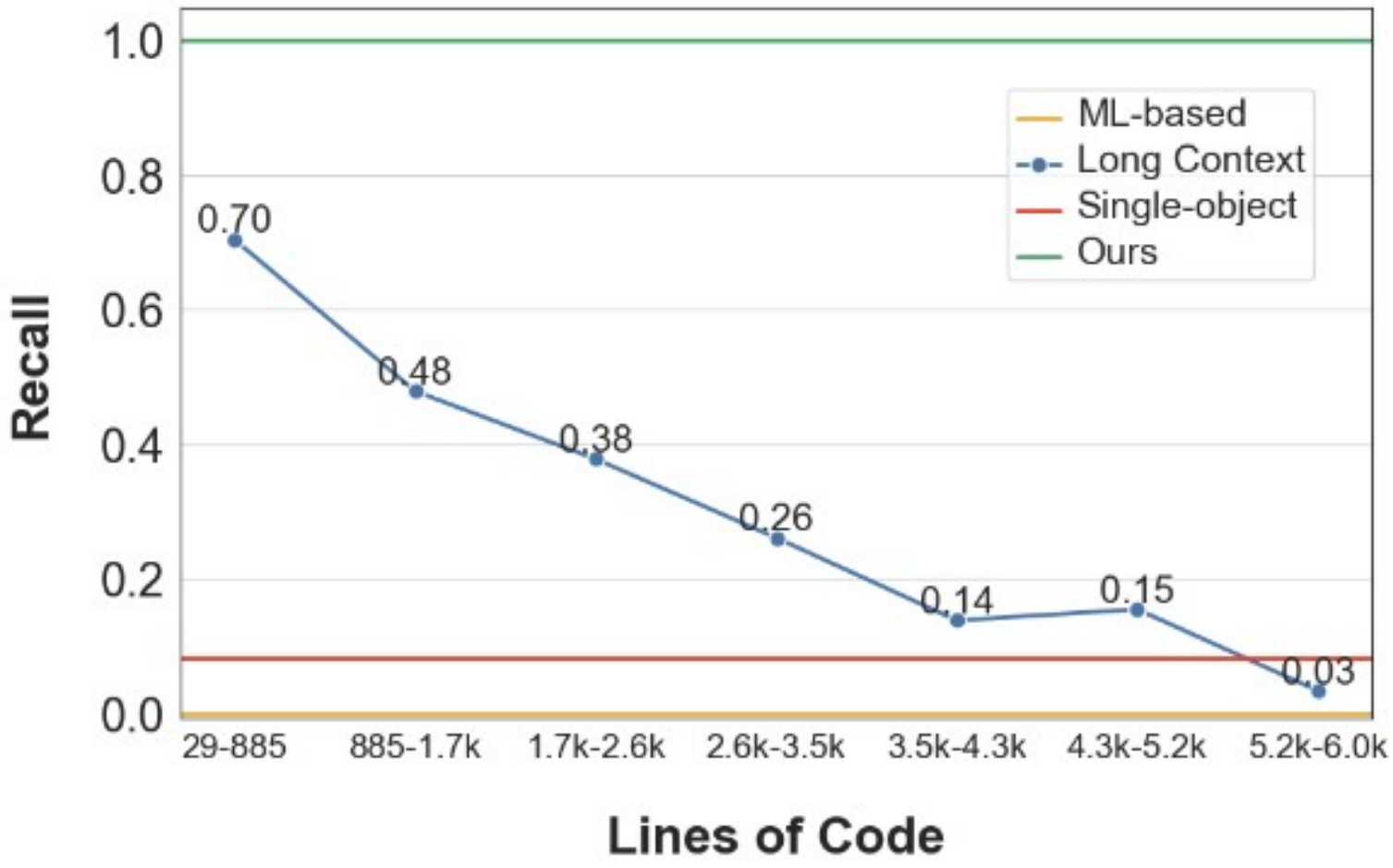
Criteria	Details
Accuracy	**Correctness**: Verify if the documentation accurately describes the code's functionality, algorithms, and expected behavior under various conditions.
	Precision: Assess whether the documentation provides precise and unambiguous information regarding the code's operations, parameters, and expected outcomes.
	Alignment with Codebase: Ensure that the documentation aligns closely with the actual implementation of the code, including any updates or changes made to the codebase.
Completeness	**Coverage**: Evaluate if the documentation comprehensively covers all significant aspects of the code, including inputs, outputs, error handling, edge cases, and any potential exceptions.
	In-depth Explanation: Determine if the documentation delves into detailed explanations of complex functionalities or algorithms, providing insights into the underlying logic.
	Documentation of External Dependencies: Check if the documentation adequately addresses any external libraries, modules, or APIs used within the codebase.
Understandability	**Clarity**: Assess the clarity and readability of the documentation, ensuring that it is easily understandable by developers of varying expertise levels.
	Conciseness: Determine if the documentation conveys information concisely without unnecessary verbosity or technical jargon that might hinder comprehension.
	Structured Organization: Evaluate if the documentation is logically organized, with clear headings, sections, and navigation aids for easy reference and comprehension.
Consistency	**Formatting Consistency**: Ensure consistency in the formatting, styling, and layout of the documentation across all sections and pages.
	Terminology Consistency: Verify that consistent terminology and naming conventions are used throughout the documentation to maintain coherence and clarity.
	Style Guide Adherence: Assess if the documentation adheres to any predefined style guides or conventions established by the project or organization.
Relevance	**Content Relevance**: Determine if the information provided in the documentation is directly relevant to the code's functionality, purpose, and usage scenarios.
	Avoidance of Redundancy: Check for redundancy or repetition within the documentation, eliminating any extraneous or irrelevant details that do not contribute to understanding the code.
Examples and Usage	**Code Samples**: Evaluate if the documentation includes sufficient code samples, snippets, or examples to illustrate the usage and implementation of key functionalities.
	Use Cases: Assess if the documentation provides real-world use cases or scenarios where the code can be applied, demonstrating its practical utility and versatility.
	Step-by-Step Instructions: Determine if the documentation offers clear, step-by-step instructions or tutorials for integrating, configuring, and utilizing the code in different environments or applications.

Reference Recall

ML-based method.traditional machine learning and deep learning methods for generating comments

Long context concatenation. directly concatenates the code snippets until the context length reaches **128k** to let the model discover reference relationships.

Single-object generation. method using LM to generate documentation by directly feeding code snippets of the target object.



Format alignment

Adherence to the format is critical in documentation generation. For certain style or consistency

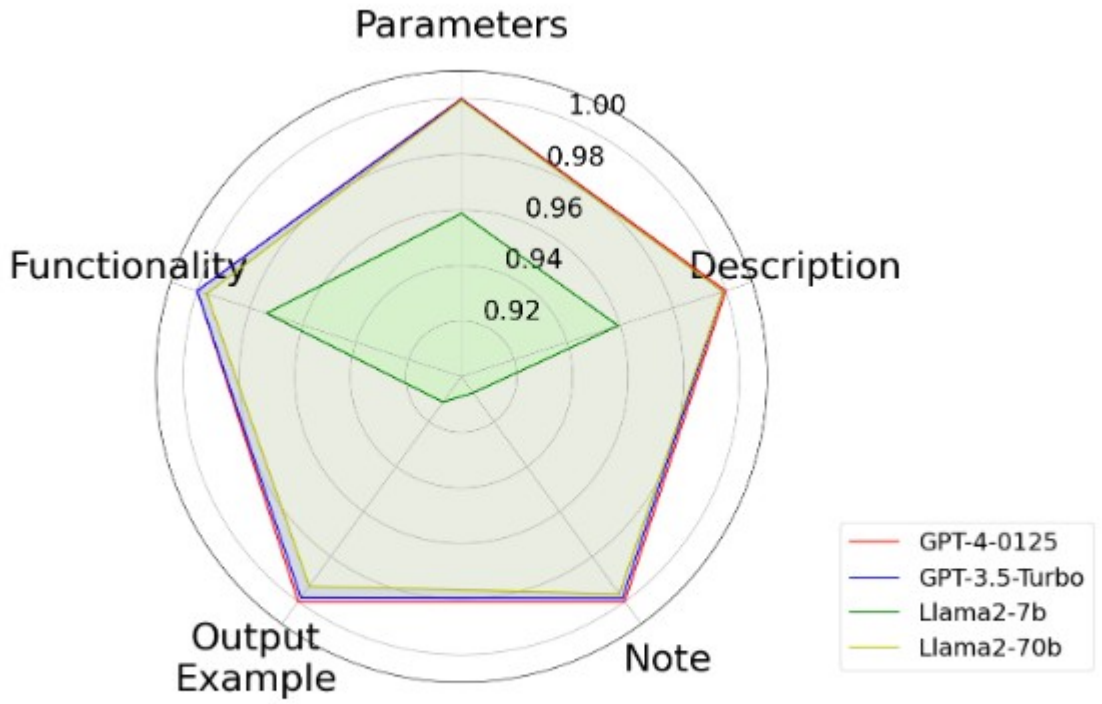


Figure 7: Format alignment accuracy of different LLMs.

Can (open-sourced) language model do that?

- Entire 5 basis sections
- Exactly the same name and format
- Can be parsed as markdown as required

from 92% to 100% accuracy on 9 repositories… they are relatively good

Parameter Identification

Note: we were calculating accuracy here, not recall.

Accurately identifying and describing parameters or attributes in code is crucial as it helps readers quickly understand the design logic and usage.

Because of hallucination !! generating nonexistent parameters or wrong affiliations

Repository	Llama-2-7b	Llama-2-70b	gpt-3.5-turbo	gpt-4-0125
unoconv	0.0000	0.5000	**1.0000**	**1.0000**
simdjson	0.4298	0.6336	**1.0000**	0.9644
greenlet	0.5000	0.7482	0.9252	**0.9615**
code2flow	0.5145	0.6171	0.9735	**0.9803**
AutoGen	0.3049	0.5157	0.8633	**0.9545**
AutoGPT	0.4243	0.5611	0.8918	**0.9527**
ChatDev	0.5387	0.6980	0.9164	**0.9695**
MemGPT	0.4582	0.5729	0.9285	**0.9911**
MetaGPT	0.3920	0.5819	0.9066	**0.9708**

a box help to focus over there

Also containing format following ability*

One more thing...

Chat with Repo

Automatic Q&A for Issues and Source Codes

query

How can I modify the update_chat_env function to track changes in the chat environment?

Instruction Editing (optional)

You are a helpful Repository-Level Software Q&A assistant. Your task is to answer users questions based on given information about a software repository, including related code and documents.

Submit

Answer:

To modify the `update_chat_env` function to track changes in the chat environment, you would need ... This could be achieved by **creating a copy of the chat environment** before the ...Here's a **simple example** of how you could modify the function:

```
def update_chat_env(self, chat_env):
    # Create a copy of the chat environment before the update
    pre_update_env = copy.deepcopy(chat_env)
    ... ... (Here is the code content omitted for presentation)
    # Compare the pre-update and post-update chat environments
    ... ... (Here is the code content omitted for presentation)
    return chat_env
```

In this example, the `copy.deepcopy` function is used to ... ...

Please note that ... ...

Code recall

1. **update_chat_env**

```
def update_chat_env(self, chat_env):
    ... ...
    return chat_env
```

2. **update_phase_env**

```
def update_phase_env(self, chat_env):
    self.phase_env.update({"task": chat_env.env_dict['task_prompt'],
                           "description": chat_env.env_dict['task_description'],
                           "ideas": chat_env.env_dict['ideas']})
```

3. **step**

```
def step(self, input_message: ChatMessage) -> ChatAgentResponse:
    ... ...
    return ChatAgentResponse(output_messages, self.terminated, info)
```

Document Recall

1. **`update_chat_env(self, chat_env) -> ChatEnv`:** This method updates the `chat_env` object based on the content of the `seminar_conclusion` attribute. If the `seminar_conclusion` contains code blocks (indicated by triple backticks), it updates the codes in `chat_env` and rewrites them with a message indicating the completion of a review cycle. It also logs software information by calling `log_visualize` with details from the `chat_env` and `log_filepath`. Finally, it updates the `phase_env` with the `seminar_conclusion` and returns the updated `chat_env`.
2. **`update_phase_env(self, chat_env)`:** This method updates the `phase_env` dictionary with information from the `chat_env` object. It extracts the task prompt, modality, ideas, language, codes, and review comments from `chat_env` and adds them to the `phase_env`. This method is crucial for ensuring that the phase environment reflects the current state of the chat environment, which is necessary for the code review and modification process.
   Code Description: The `update_phase_env` function performs the following actions:
   1. Accesses the `phase_env` attribute of the class instance (indicated by `self`) which is presumably a dictionary meant to store environment settings for a particular phase of the chat development process.
   ... ...
   4. The dictionary created in step 3 is then used to update the `self.phase_env` dictionary, effectively merging the new settings with any existing ones.

Automatic Q&A for Issues and Source Codes

query

How can I modify the update_chat_env function to track changes in the chat environment?

Instruction Editing (optional)

You are a helpful Repository-Level Software Q&A assistant. Your task is to answer users questions based on given information about a software repository, including related code and documents.

Submit

Answer:

To modify the `update_chat_env` function to track changes in the chat environment, you would need ... This could be achieved by **creating a copy of the chat environment** before the ...Here's a **simple example** of how you could modify the function:

```
def update_chat_env(self, chat_env):
    # Create a copy of the chat environment before the update
    pre_update_env = copy.deepcopy(chat_env)
    ... ... (Here is the code content omitted for presentation)
    # Compare the pre-update and post-update chat environments
    ... ... (Here is the code content omitted for presentation)
    return chat_env
```

In this example, the `copy.deepcopy` function is used to ... ...

Please note that ... ...

Code recall

1. **update_chat_env**

```
def update_chat_env(self, chat_env):
    ... ...
    return chat_env
```

2. **update_phase_env**

```
def update_phase_env(self, chat_env):
    self.phase_env.update({"task": chat_env.env_dict['task_prompt'],
                           "description": chat_env.env_dict['task_description'],
                           "ideas": chat_env.env_dict['ideas']})
```

3. **step**

```
def step(self, input_message: ChatMessage) -> ChatAgentResponse:
    ... ...
    return ChatAgentResponse(output_messages, self.terminated, info)
```

Document Recall

1. **`update_chat_env(self, chat_env) -> ChatEnv`:** This method updates the `chat_env` object based on the content of the `seminar_conclusion` attribute. If the `seminar_conclusion` contains code blocks (indicated by triple backticks), it updates the codes in `chat_env` and rewrites them with a message indicating the completion of a review cycle. It also logs software information by calling `log_visualize` with details from the `chat_env` and `log_filepath`. Finally, it updates the `phase_env` with the `seminar_conclusion` and returns the updated `chat_env`.
2. **`update_phase_env(self, chat_env)`:** This method updates the `phase_env` dictionary with information from the `chat_env` object. It extracts the task prompt, modality, ideas, language, codes, and review comments from `chat_env` and adds them to the `phase_env`. This method is crucial for ensuring that the phase environment reflects the current state of the chat environment, which is necessary for the code review and modification process.
   Code Description: The `update_phase_env` function performs the following actions:
   1. Accesses the `phase_env` attribute of the class instance (indicated by `self`) which is presumably a dictionary meant to store environment settings for a particular phase of the chat development process.
   ... ...
   4. The dictionary created in step 3 is then used to update the `self.phase_env` dictionary, effectively merging the new settings with any existing ones.

Automatic Q&A for Issues and Source Codes

query

How can I modify the update_chat_env function to track changes in the chat environment?

Instruction Editing (optional)

You are a helpful Repository-Level Software Q&A assistant. Your task is to answer users questions based on given information about a software repository, including related code and documents.

Submit

Answer:

To modify the `update_chat_env` function to track changes in the chat environment, you would need ... This could be achieved by **creating a copy of the chat environment** before the ...Here's a **simple example** of how you could modify the function:

```
def update_chat_env(self, chat_env):
    # Create a copy of the chat environment before the update
    pre_update_env = copy.deepcopy(chat_env)
    ... ... (Here is the code content omitted for presentation)
    # Compare the pre-update and post-update chat environments
    ... ... (Here is the code content omitted for presentation)
    return chat_env
```

In this example, the `copy.deepcopy` function is used to ... ...

Please note that ... ...

Code recall

1. **update_chat_env**

```
def update_chat_env(self, chat_env):
    ... ...
    return chat_env
```

2. **update_phase_env**

```
def update_phase_env(self, chat_env):
    self.phase_env.update({"task": chat_env.env_dict['task_prompt'],
                           "description": chat_env.env_dict['task_description'],
                           "ideas": chat_env.env_dict['ideas']})
```

3. **step**

```
def step(self, input_message: ChatMessage) -> ChatAgentResponse:
    ... ...
    return ChatAgentResponse(output_messages, self.terminated, info)
```

Document Recall

1. **`update_chat_env(self, chat_env) -> ChatEnv`:** This method updates the `chat_env` object based on the content of the `seminar_conclusion` attribute. If the `seminar_conclusion` contains code blocks (indicated by triple backticks), it updates the codes in `chat_env` and rewrites them with a message indicating the completion of a review cycle. It also logs software information by calling `log_visualize` with details from the `chat_env` and `log_filepath`. Finally, it updates the `phase_env` with the `seminar_conclusion` and returns the updated `chat_env`.
2. **`update_phase_env(self, chat_env)`:** This method updates the `phase_env` dictionary with information from the `chat_env` object. It extracts the task prompt, modality, ideas, language, codes, and review comments from `chat_env` and adds them to the `phase_env`. This method is crucial for ensuring that the phase environment reflects the current state of the chat environment, which is necessary for the code review and modification process.
   Code Description: The `update_phase_env` function performs the following actions:
   1. Accesses the `phase_env` attribute of the class instance (indicated by `self`) which is presumably a dictionary meant to store environment settings for a particular phase of the chat development process.
   ... ...
   4. The dictionary created in step 3 is then used to update the `self.phase_env` dictionary, effectively merging the new settings with any existing ones.

Automatic Q&A for Issues and Source Codes

query

How can I modify the update_chat_env function to track changes in the chat environment?

Instruction Editing (optional)

You are a helpful Repository-Level Software Q&A assistant. Your task is to answer users questions based on given information about a software repository, including related code and documents.

Submit

Answer:

To modify the `update_chat_env` function to track changes in the chat environment, you would need ... This could be achieved by **creating a copy of the chat environment** before the ...Here's a **simple example** of how you could modify the function:

```
def update_chat_env(self, chat_env):
    # Create a copy of the chat environment before the update
    pre_update_env = copy.deepcopy(chat_env)
    ... ... (Here is the code content omitted for presentation)
    # Compare the pre-update and post-update chat environments
    ... ... (Here is the code content omitted for presentation)
    return chat_env
```

In this example, the `copy.deepcopy` function is used to ... ...

Please note that ... ...

Code recall

1. **update_chat_env**

```
def update_chat_env(self, chat_env):
    ... ...
    return chat_env
```

2. **update_phase_env**

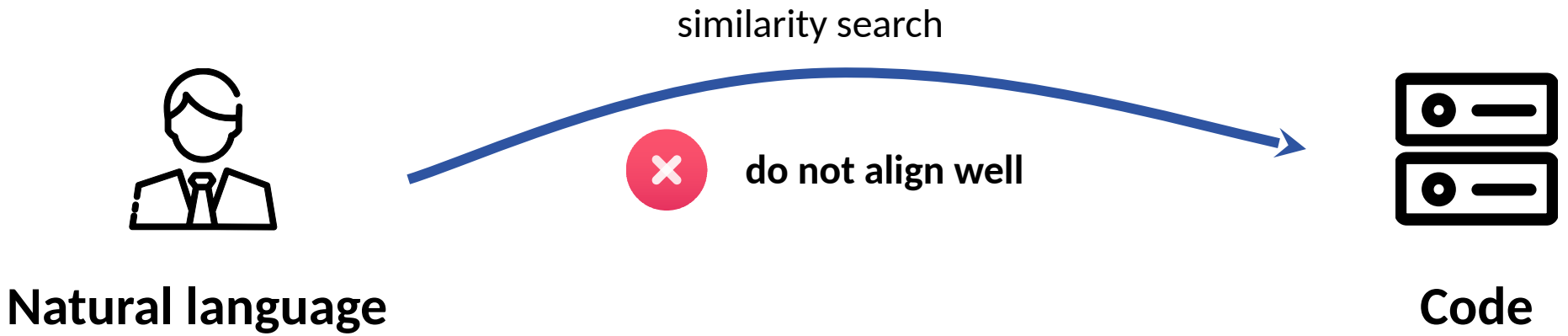
```
def update_phase_env(self, chat_env):
    self.phase_env.update({"task": chat_env.env_dict['task_prompt'],
                           "description": chat_env.env_dict['task_description'],
                           "ideas": chat_env.env_dict['ideas']})
```

3. **step**

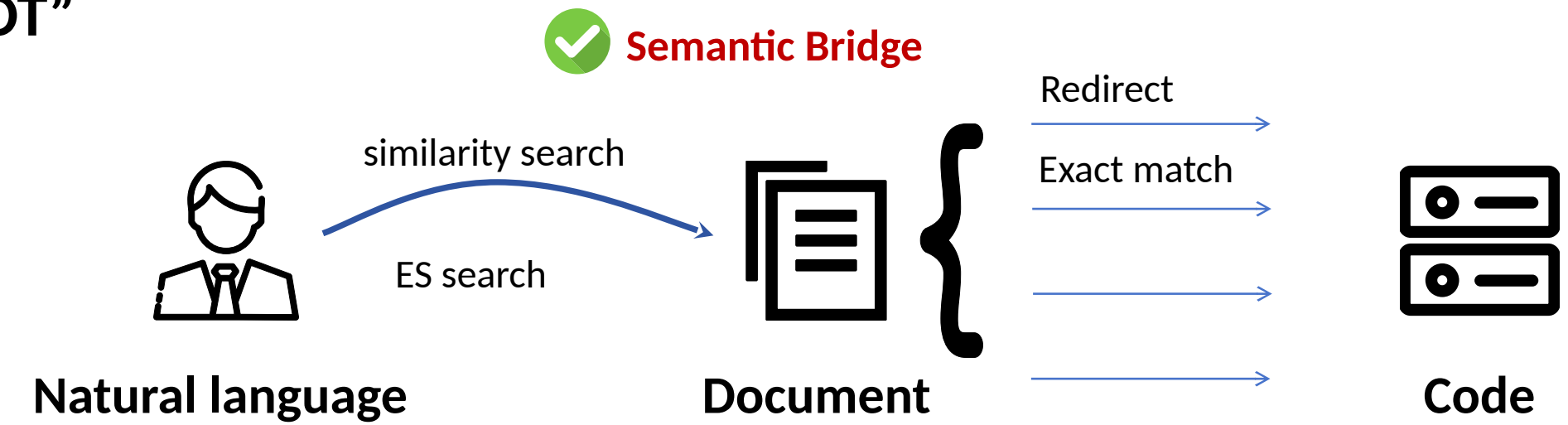
```
def step(self, input_message: ChatMessage) -> ChatAgentResponse:
    ... ...
    return ChatAgentResponse(output_messages, self.terminated, info)
```

Document Recall

1. **`update_chat_env(self, chat_env) -> ChatEnv`:** This method updates the `chat_env` object based on the content of the `seminar_conclusion` attribute. If the `seminar_conclusion` contains code blocks (indicated by triple backticks), it updates the codes in `chat_env` and rewrites them with a message indicating the completion of a review cycle. It also logs software information by calling `log_visualize` with details from the `chat_env` and `log_filepath`. Finally, it updates the `phase_env` with the `seminar_conclusion` and returns the updated `chat_env`.
2. **`update_phase_env(self, chat_env)`:** This method updates the `phase_env` dictionary with information from the `chat_env` object. It extracts the task prompt, modality, ideas, language, codes, and review comments from `chat_env` and adds them to the `phase_env`. This method is crucial for ensuring that the phase environment reflects the current state of the chat environment, which is necessary for the code review and modification process.
   Code Description: The `update_phase_env` function performs the following actions:
   1. Accesses the `phase_env` attribute of the class instance (indicated by `self`) which is presumably a dictionary meant to store environment settings for a particular phase of the chat development process.
   ... ...
   4. The dictionary created in step 3 is then used to update the `self.phase_env` dictionary, effectively merging the new settings with any existing ones.


similarity search
do not align well
Natural language
Code

“COT”


Semantic Bridge
similarity search
ES search
Redirect
Exact match
Natural language
Document
Code

Future Ideas

Repository-level-debugging
(SWE-bench or ...)

Auto testing according to document

Human in the loop

Unit Test Generation

Tutorial Generation

... ...

Thanks!

Open to any questions

Project Link

Paper

THUNLP

-- Qinyu (“Logic”) Luo Nov. 2024

Email: qinyuluo123@gmail.com
qluo8@jh.edu